

# iResolve — “Meet the Team” Page Spec

## 1. Goal

A single-page, scrollable “Meet the Team” view inside iResolve, with a **futuristic / sci-fi UI** treatment (dark mode, neon/holographic accents, glassmorphism, grid/circuit motifs) — the design itself is meant to showcase the team’s skill and polish, not list skills literally. Three tiers stacked vertically (not tabs, not separate pages). Each employee is a profile card with photo, name, and designation. Hovering a card triggers an animated popup/flip revealing **What I Do**, **Fun Fact**, and **Superpower**.

## 1.5 Visual theme — “Futuristic HUD”

- **Palette:** near-black base ( #0A0E14 ), deep navy panels ( #11182B ), one electric accent ( #00F0FF cyan or #7B5BFF violet — pick one, use sparingly), a secondary warm accent for “superpower” highlights ( #FF6B4A ), off-white text ( #E8ECF1 )
- **Type:** a geometric/technical display face for names and headings (e.g. Orbitron, Rajdhani, or Space Grotesk), a clean readable sans for body text (e.g. Inter) — don’t use the display face for paragraph text, it gets unreadable fast
- **Motifs:** thin glowing borders (1px, accent color, low-opacity outer glow), subtle animated grid/circuit-line background, scan-line or “data stream” textures used sparingly (signature element, not on every card), corner brackets on cards (like a HUD target reticle) instead of plain rounded corners
- **Glassmorphism:** detail panels use `backdrop-filter: blur()` + low-opacity background + accent-colored border, so they read as “holographic” overlays rather than flat cards
- **Restraint:** pick ONE signature futuristic effect to be the page’s memorable moment (e.g. the card border “powering on” on hover). Keep everything else quiet — glow + motion on every element at once reads as cluttered, not futuristic

## 2. Page structure (single page, tiers stacked top → bottom)

|                                    |
|------------------------------------|
| Hero / Page Title: "Meet the Team" |
|------------------------------------|

|                                    |
|------------------------------------|
| TIER 1 – Leadership (VP / AVP)     |
| [card] [card]                      |
| TIER 2 – iResolve Development Team |
| [card] [card] [card] [card]        |
| TIER 3 – Product Development Team  |
| [card] [card] [card] [card] [card] |

- Each tier is a full-width `<section>` with its own heading, on the same route/page (e.g. `/team`), in this fixed vertical order: **Leadership → iResolve Dev Team → Product Development Team**.
- Tier 1 cards can be visually larger/more prominent (leadership emphasis); Tiers 2–3 use a uniform grid.
- Smooth scroll-snap or scroll-reveal between tiers is optional polish, not required.

### 3. Data model

```
interface TeamMember {
  id: string;
  name: string;
  designation: string;
  tier: "leadership" | "iresolve_dev" | "product_dev";
  photoUrl: string;
  whatIDo: string;      // 1-2 short sentences
  funFact: string;      // 1 sentence
  superpower: string;   // 2-4 words, punchy (e.g. "Debugging at 2 AM")
}
```

Store as a static JSON/TS array for now ( `teamData.ts` ), grouped/filterable by `tier`. Easy to swap for an API call later.

### 4. Card component

**Default (resting) state:**

- Circular or rounded-square profile photo
- Name (bold, prominent)
- Designation (smaller, muted)

- Subtle idle affordance (e.g. a soft glow or small “hover to reveal” icon) so users know it’s interactive

### Hover / focus state (the “pop”):

- Triggered on `mouseenter` (desktop) **and** `onClick` / `onTap` (mobile/touch, since hover doesn’t exist there — important, don’t skip this)
- Reveals: What I Do / Fun Fact / Superpower, each as a labeled mini-block, inside a glassmorphic HUD-style panel
- Photo either flips, shrinks to a corner avatar, or stays put with a card behind it expanding — pick one consistent pattern across the whole page

Keyboard accessibility: card must be focusable ( `tabIndex=0` ) and reveal on `:focus-visible` too, not hover-only.

## 5. Animation template options (pick one direction, apply consistently)

Since the stack is React/Vite, **Framer Motion** is the natural fit (declarative, easy spring physics, good with React state). GSAP is the alternative if you want more elaborate/cinematic motion (e.g. clip-path morphs, glow trails).

### A. Holographic Flip Card

- `rotateY 180°` on hover using `transform-style: preserve-3d` + Framer Motion `whileHover`
- Front = photo/name/designation behind a faint scan-line overlay; back = glassmorphic panel with the 3 detail blocks
- Add a thin accent-colored rim light that brightens during the flip ( `box-shadow` transition) so it reads as “powering up” rather than a plain card flip
- Spring transition: `{ type: "spring", stiffness: 260, damping: 20 }`

### B. HUD Power-On Reveal

- On hover, the card’s border traces itself on (like a circuit completing) using an animated `stroke-dashoffset` SVG border, finishing in ~300ms
- Once the border completes, a frosted glass panel slides up from the bottom ( `y: "100%"` → `y: 0` ) revealing the detail text on a blurred backdrop
- Photo dims slightly and gets a subtle cyan tint overlay while the panel is up — reinforces

the “scanning” feel

- Strong fit for the Leadership tier — feels deliberate and high-production

### C. Magnetic Tilt + Holo-Expand

- Card subtly tilts toward cursor position ( `rotateX / rotateY` driven by mouse offset) using Framer Motion’s `useMotionValue + useTransform`
- On hover, card scales up slightly ( `scale: 1.05` ), accent glow intensifies, and a detail panel expands below/beside it
- More playful/energetic — scales well to the larger Dev/Product grids with many cards

### D. Staggered Data Readout (combine with any of the above)

- Detail content appears as a staggered sequence: What I Do → Fun Fact → Superpower, each block fading/sliding in with a 60–100ms delay via `staggerChildren`
- Optional: a thin animated underline or accent tick “draws itself” under each label as it appears, reinforcing the HUD/readout feel without adding extra content

**Recommendation:** Option B (HUD Power-On) for Leadership tier, Option C (Magnetic Tilt + Holo-Expand) for the two larger dev/product grids. Use Option D’s staggered readout inside both — it’s the detail that sells “futuristic” without needing more visual noise.

## 6. Page-load entrance animation

- Tiers reveal on scroll using `whileInView` (Framer Motion) — each tier’s cards fade + slide up ( `y: 20 → 0` , `opacity: 0 → 1` ) with a small stagger per card as it enters the viewport.
- Hero title can do a one-time letter/word stagger-in on initial mount, optionally with a brief glitch/flicker effect (1–2 frames) for the futuristic feel — keep it under ~400ms total, a long glitch reads as broken rather than intentional.
- Optional background: a faint animated grid or slow-drifting particle field behind the whole page (low opacity, doesn’t compete with card content) — pure CSS/canvas, keep it performance-light.

## 7. Tech recommendations

- **React + Framer Motion** (matches existing iResolve React/Vite frontend — no new heavy dependency)

- Tailwind CSS for layout/grid ( `grid-cols-2 md:grid-cols-4` etc. per tier)
- Respect `prefers-reduced-motion` : fall back to instant show/hide of detail content, no flip/tilt physics, for users with that OS setting

## 8. Suggested component breakdown

```
/team (public – people surface, read-mostly engagement reflection)
├ TeamPage.tsx           (page shell, hero, activity ticker, renders 3 <Team
├ TeamTier.tsx           (tier heading + grid of <TeamCard>)
├ TeamCard.tsx           (card + hover/flip/tilt logic + kudos badge)
├ ModulesActivityBlock.tsx (compact per-developer modules + latest comment
├ ActivityTicker.tsx      (site-wide live activity marquee, shared with /
└ teamTypes.ts           (TeamMember, Module, Comment, Appreciation inte
```

```
/modules (public – work surface, where likes & comments actually happen)
├ ModulesHubPage.tsx      (searchable/filterable module grid)
├ ModuleCard.tsx          (grid item: name, owner, tag, status badge, like
├ ModuleDetailPage.tsx    (full module page: description, Like button, Co
├ LikeButton.tsx          (reaction + burst animation)
└ CommentsSection.tsx     (thread + submit form)
```

```
/admin/settings/team (Super Admin only)
├ TeamAdminPage.tsx       (team member management table: list/add/edit/dele
├ TeamMemberForm.tsx      (add/edit form, photo upload)
├ ModuleAdminPage.tsx     (module management: add/edit/delete/assign owne
└ CommentModerationQueue.tsx (pending comments: approve/hide)
```

```
Backend (FastAPI)
├ GET /api/team            (public, read-only)
├ GET /api/team/{id}/activity (public, modules + like counts + latest
├ GET /api/modules        (public, list/search/filter)
├ GET /api/modules/{id}    (public, module detail incl. comments)
├ GET /api/activity/recent (public, powers the ticker)
├ POST /api/modules/{id}/appreciate (auth required)
├ POST /api/modules/{id}/comments (auth required)
├ POST /api/admin/team      (Super Admin only)
├ PUT /api/admin/team/{id}  (Super Admin only)
├ DELETE /api/admin/team/{id} (Super Admin only)
├ PUT /api/admin/team/reorder (Super Admin only)
├ POST /api/admin/modules   (Super Admin only)
├ PUT /api/admin/modules/{id} (Super Admin only)
└ PUT /api/admin/comments/{id}/status (Super Admin only – approve/hide)
```

## 9. Admin management (Super Admin user group only)

Team data should not live as a static file — it needs to be editable without a code deploy. This moves it into iResolve's existing PostgreSQL + FastAPI stack and reuses the existing permission system.

### Data layer

- New `team_members` table in PostgreSQL: `id`, `name`, `designation`, `tier`, `photo_url`, `what_i_do`, `fun_fact`, `superpower`, `sort_order`, `created_at`, `updated_at`
- `teamData.ts` goes away; `TeamPage` fetches from a `GET /api/team` endpoint instead (public, read-only, used by the live page)

### Admin API (write access gated to Super Admin)

- `POST /api/admin/team` — add employee
- `PUT /api/admin/team/{id}` — edit employee details
- `DELETE /api/admin/team/{id}` — remove employee
- `PUT /api/admin/team/reorder` — update `sort_order` (so admins can drag-reorder within a tier)
- Reuse the existing `require_module()` FastAPI dependency pattern, scoped to a `super_admin` permission/module so only that AD/LDAP group can hit these routes — same approach as the rest of iResolve's database-driven per-user module permissions.

### Admin UI

- New page under Admin Settings, e.g. `/admin/settings/team`, visible only to users in the Super Admin group (hide the nav entry entirely for everyone else, don't just block the route)
- A simple management table/grid: photo thumbnail, name, designation, tier — with Add Employee / Edit / Delete actions
- Edit/Add form fields: Name, Designation, Tier (dropdown: Leadership / iResolve Dev / Product Dev), Photo (upload, with a sensible max size + crop-to-square or fixed aspect ratio), What I Do, Fun Fact, Superpower
- On save, the public `/team` page reflects changes immediately (no cache, or a short cache TTL) — no rebuild/redeploy needed
- Optional but recommended: confirm-before-delete modal, since removing an employee is destructive

**Out of scope for v1:** version history / audit log of who changed what — flag as a possible v2 if needed later.

## 10. Site-wide “Working” Indicator (Claude-Code-style loading states)

This isn’t just for the Meet the Team page — it’s a reusable pattern for the whole iResolve webpage, anywhere a get-count, validation run, or reconciliation job is in progress and the user needs reassurance it hasn’t frozen. Same idea as Claude Code’s playful status words while it works, applied to iResolve’s own long-running operations.

### 10.1 Where it’s used

- Any async operation with no instant result: account count pulls, ad-hoc letter eligibility runs, reconciliation jobs, BRD interpretation, QA test runs, file-server date-range reads — basically anywhere the user currently just sees a blank state or a generic spinner today.

### 10.2 Behavior

- A thin horizontal bar (matches the futuristic theme — glowing scan-line animation moving left-to-right, accent color) paired with a single rotating status word/phrase + animated ellipsis (e.g. “Reconciling...”)
- The word swaps every ~1.2–1.8s, picked randomly from the word bank below, never repeating the same word twice in a row
- After a longer threshold (e.g. 20–30s), switch to a steadier reassurance state — e.g. “Still working — large jobs can take a few minutes” — so a long reconciliation doesn’t feel like it’s stuck in a joke loop
- Respect `prefers-reduced-motion` : keep the word rotation (it’s informational) but drop the moving scan-line animation in favor of a static glow

### 10.3 Implementation notes

- Build as one shared component (e.g. `<WorkingIndicator />` + a `useWorkingMessage()` hook) so every module across iResolve reuses it instead of each team rolling their own spinner
- `aria-live="polite"` on the status text so screen readers announce updates without being spammed on every word change
- Word bank lives as a simple array; component does `pick random, not equal to last`

shown on each interval tick

## 10.4 Word bank (mix of playful generic + a few iResolve-flavored ones for personality)

Reticulating...  
Pontificating...  
Marinating...  
Percolating...  
Synthesizing...  
Triangulating...  
Conjuring...  
Noodling...  
Tinkering...  
Crunching...  
Untangling...  
Calibrating...  
Spelunking...  
Divining...  
Sleuthing...  
Orchestrating...  
Deliberating...  
Cogitating...  
Frobnicating...  
Wrangling the bytes...  
Polishing the numbers...  
Summoning the results...  
Squaring the ledger...  
Counting the accounts...  
Untangling Artiva...  
Chasing stray decimals...  
Auditing the audit...  
Querying the matrix...  
Balancing the books...  
Consulting the abacus...

That's 30 — trim or extend as you like, but keep the iResolve-flavored ones (ledger/accounts/Artiva/decimals) since they give it personality specific to what the tool actually does, rather than reading as a generic borrowed gimmick.

## 11. Social / Engagement Layer — “Module Appreciation & Feedback”

The goal: turn the page from a static directory into something end users actually return to.



Likes and comments live on the **Module** itself — modules get their own hub/detail page where the actual interaction happens — and the Meet the Team page is a live, read-mostly reflection of that activity on each developer's card.

## 11.0 Where engagement actually lives

- New top-level **Modules Hub** (e.g. `/modules`) — a directory of everything shipped inside iResolve: card grid, searchable/filterable by tag (e.g. "ETL", "Auth", "Admin Tooling")
- Each module has its own **detail page** (`/modules/{id}`) — description, owner(s), status badge (Live / Beta / Deprecated), and this is where the **Like** button and **Comments section** actually live, same way a real product page or a GitHub repo would host them
- The Meet the Team page does **not** duplicate the full comment box — it shows a live-updating summary (kudos count, latest comment snippet) per developer, pulled from their modules, with a "View on Modules Hub" link that deep-links straight to the module's comment thread for the full interaction
- This keeps Meet the Team focused on the people, and Modules Hub focused on the work — but they're wired together so activity on one shows up on the other in near-real-time

### 11.1 Core concept

- A **Module** is a thing a developer shipped/owns inside iResolve (e.g. "Ad-hoc Letter Eligibility", "File Server Date Reader"), linked to one or more team members.
- End users (any logged-in iResolve user, reusing existing Windows Auth/LDAP identity — no separate signup) can, **from the module's own page**:
  - **Like/Appreciate** it with a quick reaction (Instagram-style — tap an icon, get a burst animation, count increments)
  - **Comment** — a short message tied to the module, visible in its comment thread
- That activity rolls up onto the owning developer's card on the Meet the Team page, live.

### 11.2 Data model additions

modules

```
id, name, description, tag, status, owner_member_id (FK -> team_members), creat
-- tag: free-text or small fixed taxonomy, e.g. "ETL", "Auth", "Admin Tooling"
-- status: "live" | "beta" | "deprecated"
```

appreciations

```
id, module_id (FK), user_id (from existing auth), reaction_type, created_at
```

```
-- reaction_type: "👏" | "🔥" | "💡" | "🚀" (small fixed set, not free-for-all)

comments
  id, module_id (FK), user_id (from existing auth), message, status, created_at
  -- status: "pending" | "approved" | "hidden" (for moderation, see 11.6)
```

- appreciations should have a unique constraint on (module\_id, user\_id, reaction\_type) — one of each reaction per user per module, not infinite spam-clicking.

## 11.3 API

- GET /api/modules — list/search/filter modules for the Modules Hub
- GET /api/modules/{id} — module detail: description, status, owner(s), like count, approved comments
- POST /api/modules/{id}/appreciate — toggle a reaction on the module page (auth required, rate-limited)
- POST /api/modules/{id}/comments — post a comment on the module page (auth required, goes to pending — see 11.6)
- GET /api/team/{id}/activity — returns a member's modules + aggregated like counts + latest 1–2 approved comment snippets, used to populate their Meet the Team card (read-only reflection, not a write surface)
- GET /api/activity/recent — paginated feed of recent likes/comments across all modules, powers the live ticker on both pages

## 11.4 UI behavior

### On the Modules Hub / module detail page (where the actual interaction happens):

- Module detail page has a clear Like button (with burst animation + count) and a full Comments section underneath — standard, recognizable pattern, nothing exotic needed here since this is the “work” surface
- Status badge (Live / Beta / Deprecated) styled with the futuristic theme's glow language

### On the Meet the Team page (the “people” surface, read-mostly reflection):

- Each card gets a small **kudos badge** in its resting state (e.g. a glowing counter — “🔥 128”, summed across that person's modules) so engagement is visible without even hovering
- In the hover/expanded detail panel, add a compact **Modules & Activity** block: the

developer's modules listed with mini like-counts, and the single latest approved comment as a snippet (like an Instagram comment preview) — tapping it deep-links straight to that module's full page/thread

- A site-wide **live activity ticker** (thin marquee strip, e.g. just below the hero) shared by both pages, showing real-time-ish activity: "Priya's File Server Date Reader just got a 🔥" — adds ambient life even for users just browsing

## 11.5 Live updates

- Simplest reliable approach: poll `GET /api/activity/recent` and the visible cards' activity every 15–30s and animate in any deltas (count ticks up with the same burst animation, ticker gets a new line)
- If real-time truly matters later, this can upgrade to Server-Sent Events or a WebSocket without changing the data model — note this as a v2 swap, not a v1 requirement

## 11.6 Moderation & abuse prevention

- Comments default to `pending` and only show publicly once `approved` — Super Admins moderate from the same Admin Settings area as Section 9 (a "Pending Comments" queue: approve / hide)
- Rate-limit likes and comment submission per user (e.g. max N comments/day) to prevent spam
- Since users are authenticated via existing Windows Auth/LDAP, comments aren't anonymous to the system even if displayed without a name — gives a moderation trail without extra infra

## 11.7 More ideas worth considering (not required for v1, flagging for your call)

### Gamification / recognition

- **"Most Appreciated This Month"** badge that auto-highlights the top card per tier
- **Streak/achievement badges** for developers (e.g. "Shipped 3 modules this quarter", "Top Responder", "100 Club") shown as small icons next to their name
- **Hall of Fame strip** at the bottom of Modules Hub — top 3 most-liked modules this month, rotating

**Visual/interaction** — leans into the futuristic theme rather than bolting engagement on top of it

- **Skill constellation map**: instead of (or alongside) the card grid, an optional alternate

view of the team page as an interactive node-link graph — developers as glowing nodes, modules as connecting lines, hovering a node highlights what they've shipped. Pure sci-fi HUD energy, and it visually *shows* capability across the org rather than telling

- **Reaction-triggered card glow:** when a card crosses an appreciation threshold (e.g. 100), its border glow upgrades to a more intense animated state
- **"Tap to like" double-tap on a module's hero image** (literal Instagram gesture) as an alt path to the Like button on the module detail page

## Content & discovery

- **Module changelog/timeline** on the detail page — lightweight version history ("v1.2 — added bulk export") so the module page feels alive even between comments
- **Weekly digest:** each developer (or Super Admin) gets a quiet auto-summary — "Your module got 12 likes and 3 comments this week" — could reuse the Claude API already in iResolve's stack to auto-draft a one-line friendly recap instead of a raw stat dump
- **@mentions in comments:** a commenter can @mention a developer, which surfaces in that developer's admin/dashboard view as a lightweight notification — no email infra needed if it just lives in-app

## Self-service

- **"Submit a module" flow:** developers add their own modules from a simple form (name, description, tag) which goes to Super Admin for approval before appearing on the Hub — keeps the directory accurate without Super Admin manually entering everything, while still gating what goes public
- **Comment reactions:** a lightweight 👍/❤️ on individual comments (not full reaction set) — cheap to add once the base Like infra exists, and gives the comment thread its own bit of life

**Restraint note:** not all of these should ship in v1 — the constellation map and changelog in particular are bigger builds. Worth picking 1–2 from this list that feel most "us" rather than building all of them at once.

## 12. Acceptance criteria

- All 3 tiers render stacked vertically on one page, in fixed order (Leadership → iResolve Dev → Product Dev)
- Each card shows photo, name, designation by default
- Hover (desktop) and tap (mobile) reveal What I Do / Fun Fact / Superpower with smooth

animation

- Keyboard-focusable cards also trigger the reveal
- Reduced-motion users get a non-animated fallback
- Responsive: grid collapses gracefully on mobile (1–2 columns)
- Team data is DB-backed ( `team_members` table); Super Admin group can add/edit/delete employees and reorder within a tier from Admin Settings; changes reflect on the public page without a redeploy
- Admin Settings team page is hidden from non-Super-Admin users, not just route-blocked
- Modules Hub exists at `/modules` with a searchable grid and a detail page per module; Like button and Comments section live on the module detail page
- Meet the Team cards show a live, aggregated kudos count and latest comment snippet per developer, sourced from their modules — no separate like/comment input on the Team page itself, just a deep-link to the module
- Comments can be submitted by authenticated users on a module's page, default to pending, and only approved comments show publicly (on both the module page and the Team page reflection)
- Super Admin has a pending-comments moderation queue (approve/hide) and can manage modules (add/edit/assign owner) from Admin Settings
- Activity (likes + approved comments) updates on both pages without a manual refresh (polling is sufficient for v1)
- A shared `<WorkingIndicator />` component exists and is used for at least one real async operation (e.g. account count, reconciliation run) site-wide, with rotating status words, a long-running fallback message, and reduced-motion support